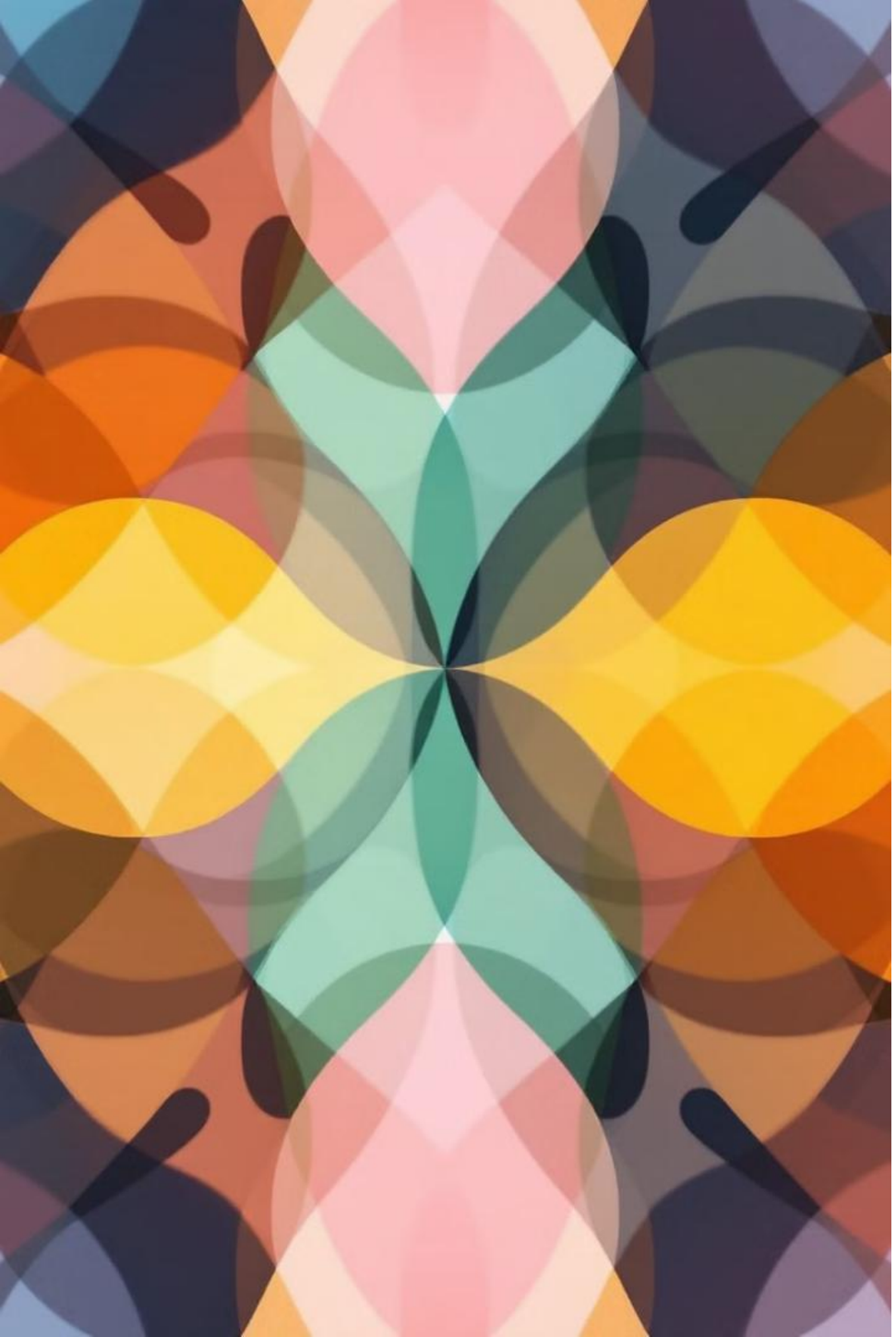


🚀 Optimizing List Performance in React Native

Why List Optimization Matters?

Slow Rendering Issues

Large lists in React Native can pose performance challenges, leading to slow rendering times and user frustration. This is particularly noticeable when dealing with dynamic data, frequent updates, or complex list items.

Real-World Examples

Imagine scrolling through a social media feed with thousands of posts. If the app isn't optimized, you might experience lag, delayed content updates, or even a complete freeze of the UI. This can significantly impact user experience and engagement.

What is Virtualization?

1 Definition

Virtualization is a technique that renders only the list items currently visible on the screen, instead of rendering all items at once. This significantly reduces the amount of data and UI elements that need to be processed, leading to improved performance.

2 Benefits

By rendering only visible items, virtualization offers several key advantages:

- Enhanced performance: Faster rendering and smoother scrolling.
- Reduced memory consumption: Lower memory footprint, crucial for large datasets.
- Prevents UI lag: Smooth user experience, even with frequent data updates.

How Virtualization Works?

Windowing Technique

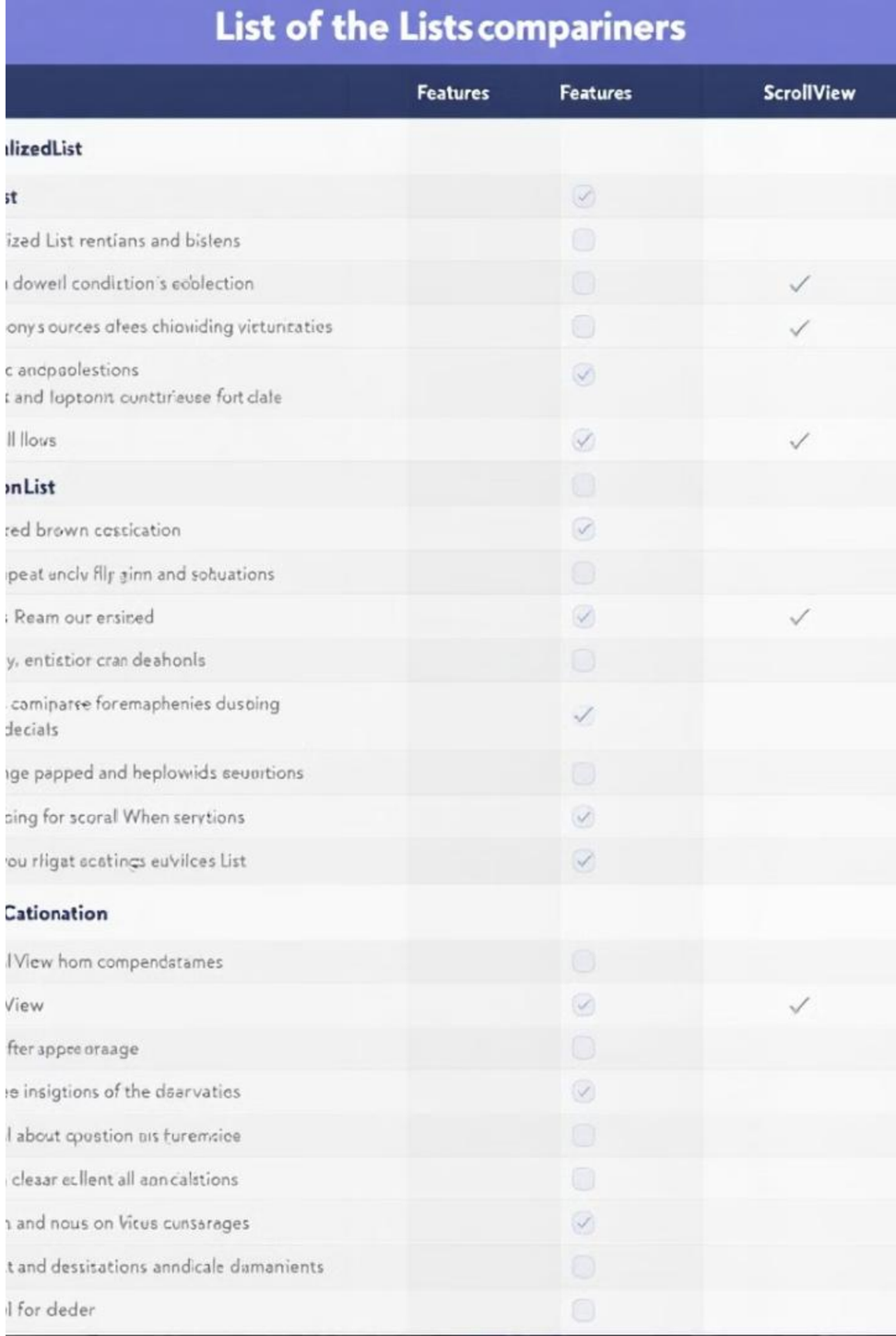
Virtualization relies on a windowing technique. It divides the list into smaller "windows," typically based on the screen's viewport. Only the items within the visible window are rendered, while the rest remain hidden.

Illustration

Imagine a list with 100 items. Before virtualization, all 100 items would be rendered. After virtualization, only the items visible within the current screen window would be rendered. As the user scrolls, the window shifts, and new items are rendered as necessary. This dynamic rendering significantly optimizes performance.

Different List Components

VirtualizedList	Yes	Excellent (with optimizations)	Large & Dynamic Lists
FlatList	Yes (default)	Good (default)	Medium-Sized Lists
SectionList	Yes (default)	Good (default)	Lists with sections
ScrollView	No	Can be slow with large lists	Small, static lists



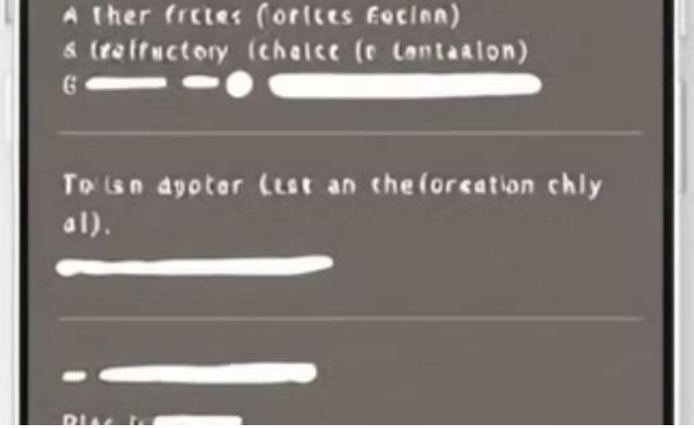
Understanding RCTScrollView

React Native's Scrolling Foundation

At the core of React Native's scrolling capabilities is RCTScrollView. This native component provides the foundational infrastructure for efficient scrolling, enabling the smooth rendering of lists and other scrollable content.

How RCTScrollView Powers Lists

Different list components like VirtualizedList, FlatList, and SectionList build upon the functionality of RCTScrollView. They leverage its optimized scrolling mechanisms to handle the rendering and management of lists, providing a seamless user experience.



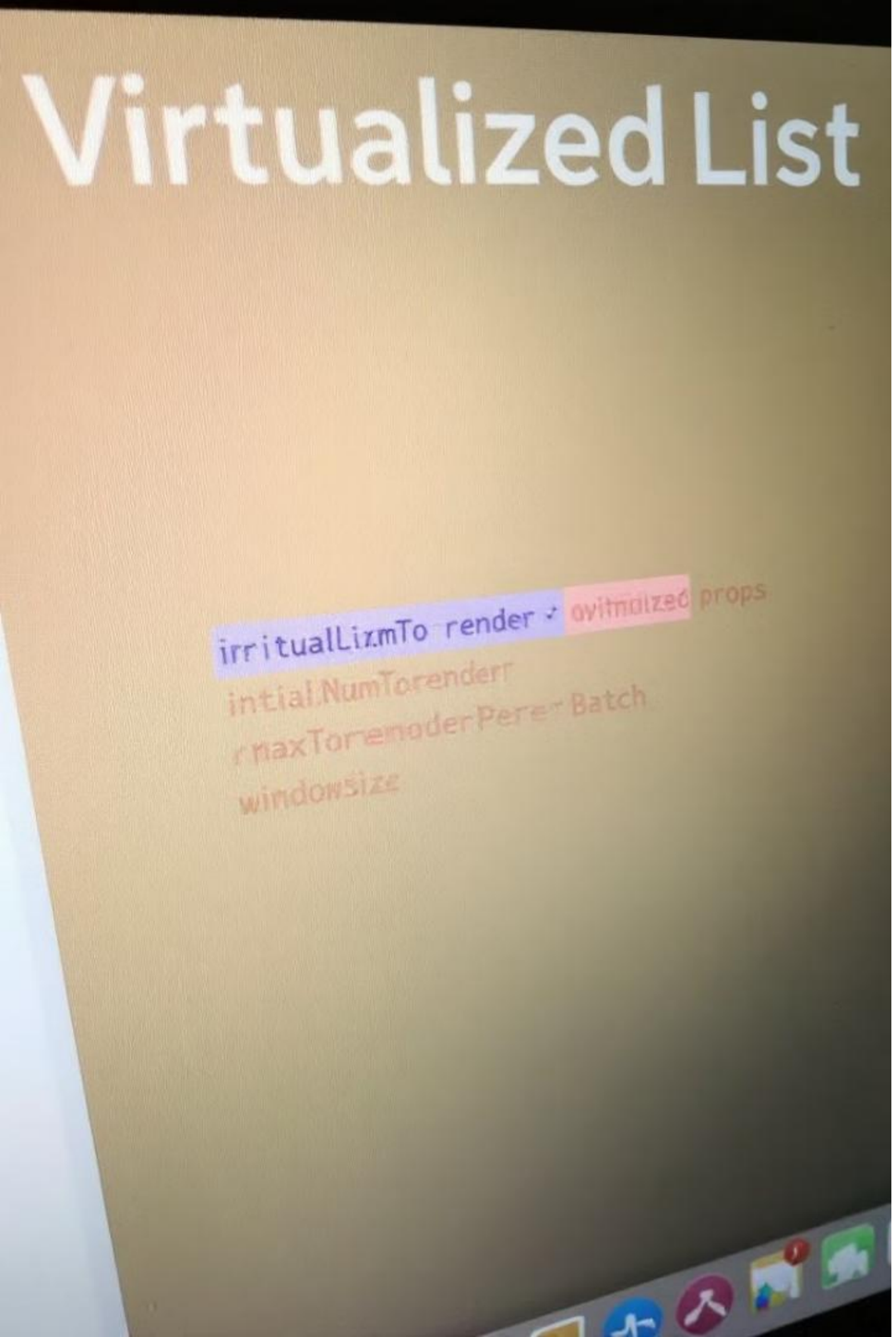
Basic VirtualizedList Implementation

```
<VirtualizedList
  data={DATA}
  initialNumToRender={10}
  renderItem={({ item }) => <Text>{item}</Text>}
  keyExtractor={(item, index) => index.toString()}
  getItem={(data, index) => data[index]}
  getItemCount={(data) => data.length}
/>;
```

This code demonstrates a basic VirtualizedList implementation. Key props include data, initialNumToRender, renderItem, keyExtractor, getItem, and getItemCount. It initializes with 10 items and dynamically renders list items based on the data provided.

https://snack.expo.dev/@taitv/virtualizedlist

```
// Virtualized List Render JSX element 👇
function renderVirtualizedListItem({ item, index }) {
    return (
        <View key={index} style={styles.virtualizedListStyle}>
            <TouchableOpacity
                onPress={async () => {
                    setNftDetails(item)
                    setPlayerCardModal(true)
                }}
                style={{
                    ...styles.nftCard
                }}>
                <Image
                    style={styles.image}
                    resizeMethod="resize"
                    source={{
                        uri:
                            item.collectionType == NFT_COLLECTION_TYPE.FEMALE
                                ? item.easyImageUrl
                                : item.url
                    }}
                />
                <Text style={{ ...FONTS.regular12 }}>{item.nftName}</Text>
            </TouchableOpacity>
        </View>
    )
}
```



Optimizing VirtualizedList Performance

```
<VirtualizedList
  data={DATA}
  initialNumToRender={10}
  maxToRenderPerBatch={10}
  windowSize={5}
  removeClippedSubviews={true}
/>;
```

Key Optimization Props

VirtualizedList offers a set of props designed to fine-tune its performance.

- initialNumToRender: Defines the initial number of items to render when the list is mounted.
- maxToRenderPerBatch: Limits the number of items rendered in each batch as the user scrolls.

- windowSize: Determines the number of items rendered around the visible window.
- removeClippedSubviews: Removes off-screen items from the view hierarchy, reducing memory overhead.

Challenges with VirtualizedList

Common Problems

While VirtualizedList offers excellent performance, there are common challenges that developers might encounter:

- Jittery scrolling: Unpredictable scroll behavior, particularly when data is being updated.
- Blank items during fast scrolling: Gaps or blank spaces appearing while scrolling quickly.
- High memory usage on low-end devices: Excessive memory consumption, especially with large datasets.

Solutions

Addressing these challenges often involves fine-tuning virtualization settings and implementing best practices:

- Adjust windowSize: Fine-tune the number of items rendered around the visible window to balance performance and accuracy.
- Implement lazy loading: Load data on demand as the user scrolls, preventing initial memory overhead.
- Optimize renderItem function: Keep the logic within the renderItem function concise and efficient, minimizing rendering time.

FlatList vs VirtualizedList

Virtualization	Yes	Yes (default)
Performance	Excellent (with optimizations)	Good (default)
Best For	Large, dynamic lists	Medium-sized lists, less demanding scenarios
Customization	High	Medium
Learning Curve	Steeper (more options)	Gentle (simpler usage)

While both VirtualizedList and FlatList provide virtualization, they cater to different use cases. For large, dynamic lists requiring fine-grained performance optimization, VirtualizedList is preferred. FlatList is generally suitable for medium-sized lists and scenarios where default virtualization settings are sufficient. Choosing the right component is crucial for optimal list performance and user experience.

What is moust on React Native table irst?

VirtualizedList	Flat List
✔ Commanzen	✔ Comprionide liine
✔ Nutranization	✔ Autraijzation
✔ Advlinagedlisterfor fecation	✔ Exeepingedetam acfcer ceutisn
✔ Companti apacllity recebization.	✔ Computting fectiges
✔ Complonless ceatues	✔ Commonsicaaton
✔ Mayonaliaables port scautiod	✔ Mugna d wiscnt seaturs
✔ Plepertien	✔ Fleatural
✔ Conptiondincenecation pertions	✔ Cemrization clures incenseration
✔ Dempllivizes	✔ Disecfity
✔ Duntaliz and ren craust	✔ Nentalistycliphen cart
✔ Heyrding benefits	✔ Expming an lulrtced
✔ Aduitiveafinactinations	✔ Accepuratimatabls ver secation
✔ Durminties of fors cetarses	✔ Cutt ters ther kny initiallage for allonr direcations
✔ Fullancation edlircation	✔ Fullr nuation marvices

Understanding FlatList: The Foundation of List Rendering

What is FlatList?

FlatList is React Native's primary component for rendering long lists of data efficiently. It's designed to handle a large number of items without causing performance issues.

Key Features

Virtualization: Renders only visible items for optimal performance. Flexibility: Supports horizontal scrolling and customization with props. Efficiency: Optimizes memory usage and prevents UI lags.

Essential FlatList Props

1 data

An array of data representing items in the list.

2 renderItem

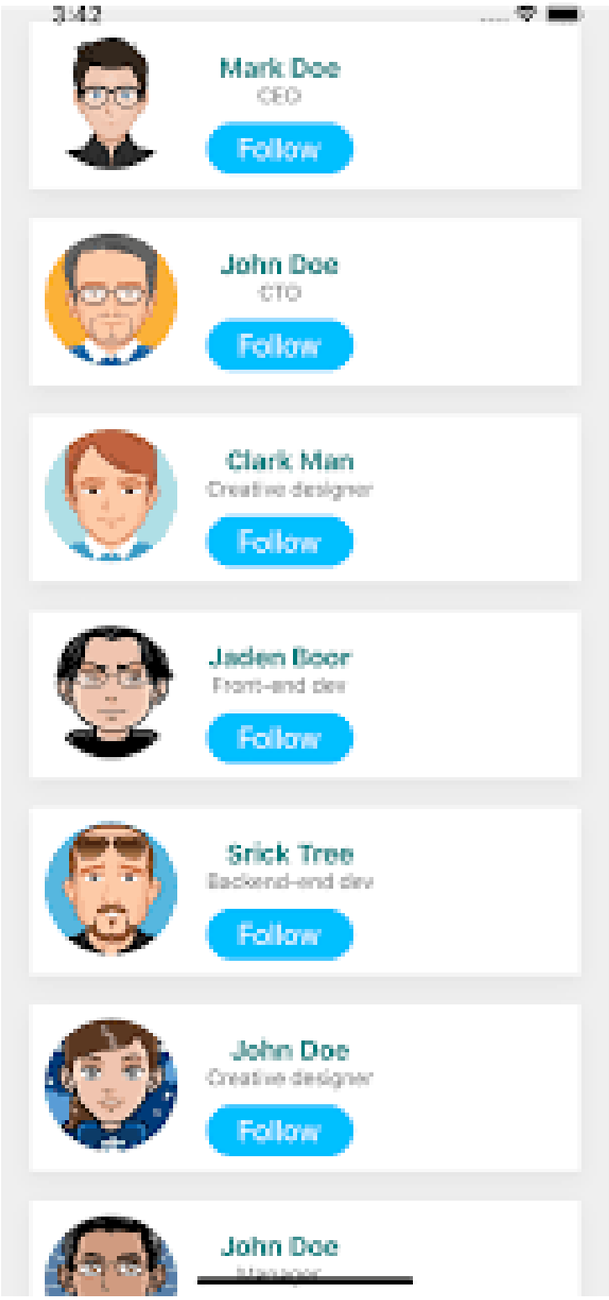
A function that renders each item in the list.

3 keyExtractor

A function that provides a unique key for each item in the list.

4 getItemLayout

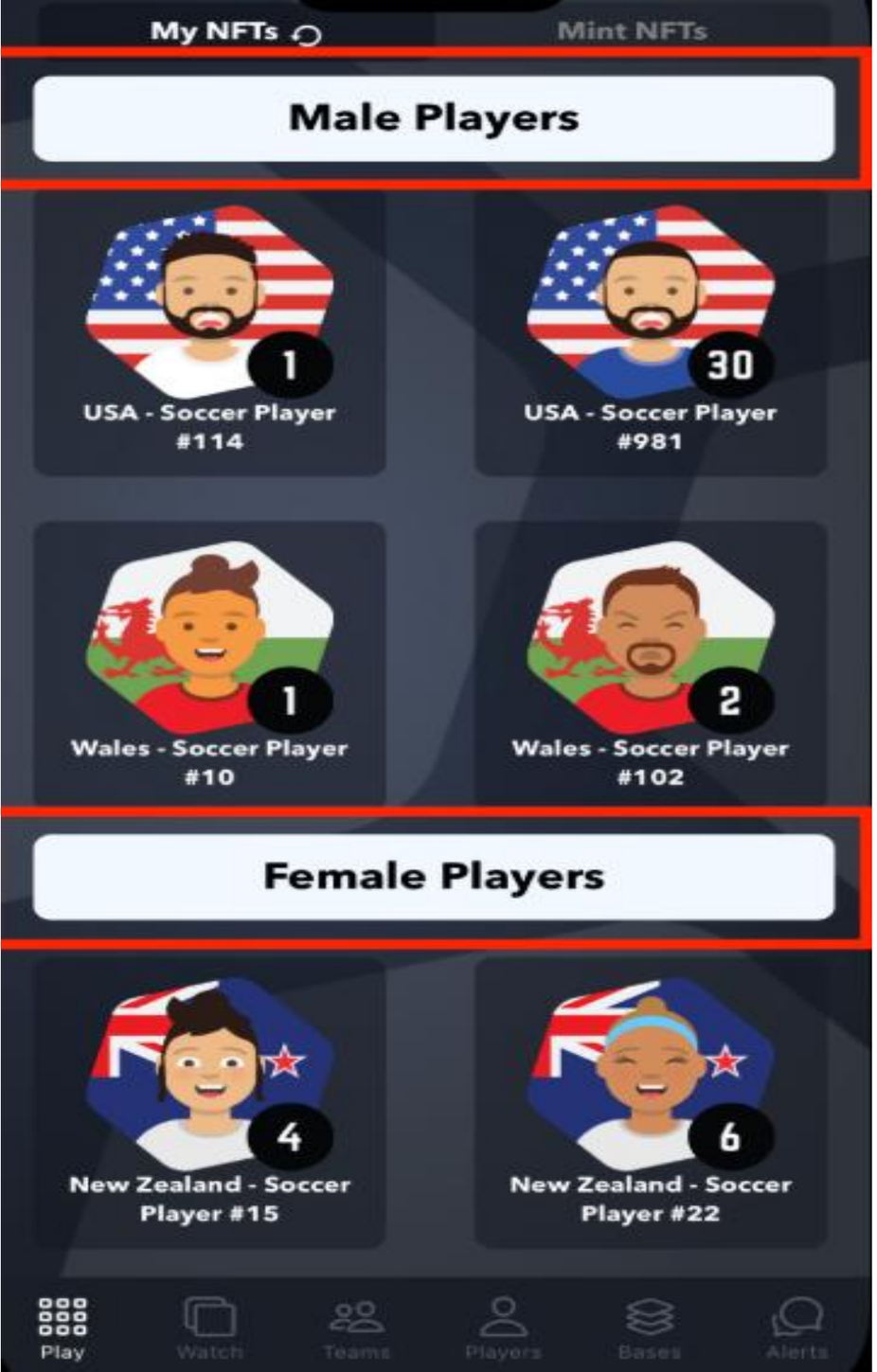
A function that tells FlatList how to measure item dimensions and position in the list.



FlatList Performance Optimization

```
<FlatList
    data={myNftsData.findMyNfts}
    renderItem={renderFlatlistItem}
    keyExtractor={(item) => item._id}
    numColumns={2}
    removeClippedSubviews={true}
    maxToRenderPerBatch={10}
    updateCellsBatchingPeriod={100}
    initialNumToRender={10}
    windowSize={11}
/>
```

https://snack.expo.dev/@taitv/2f442d



SectionList: Grouped Lists Made Easy

What is SectionList?

A component for rendering lists with sections, useful for grouping similar items together.

Typical Use Cases

Contacts (grouped by initials), Products (grouped by category), Calendar events (grouped by date).

Performance Considerations

Uses the same principles of virtualization and efficiency as FlatList.

Implementing SectionList

Code Example

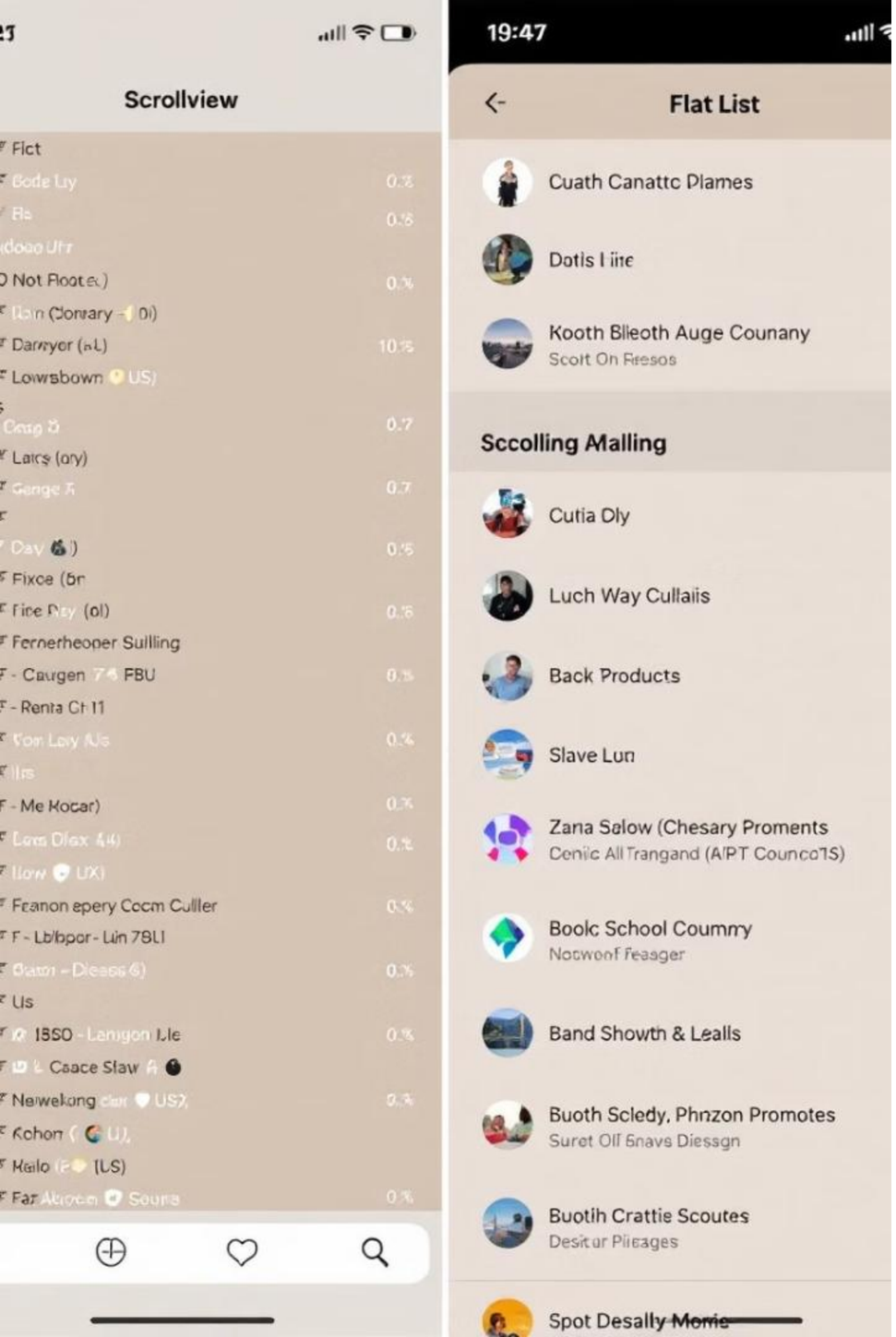
```
<SectionList
    sections={sectionsData()}
    renderItem={renderSection}
    renderSectionHeader={renderSectionHeader}
    keyExtractor={(item) => item._id}
/>
```

Key Points

Provide an array of sections, each with a title and data array. Use renderItem and renderSectionHeader for customizing item and section header rendering.

SectionList Performance Optimization

```
return (
  <SectionList
      sections={sectionsData()}
      renderItem={renderItem}
      renderSectionHeader={renderSectionHeader}
      keyExtractor={(item) => item._id}
      // Performance props
      maxToRenderPerBatch={20} // Increase the number of items per batch
      updateCellsBatchingPeriod={100} // Increase the time interval between updates
      windowSize={11} // Reduce the number of screens to render
      removeClippedSubviews={true} // Remove offscreen views
    />
)
```



Why Avoid ScrollView for Large Lists?

1

ScrollView

Renders all items at once, leading to slow performance and high memory consumption.

2

FlatList/VirtualizedList

Only render visible items, optimizing performance and memory usage.

Advanced Optimization Techniques

1 Pagination

Load data in chunks as the user scrolls, reducing initial loading time and memory pressure.

2 React.memo

Prevent unnecessary re-renders of list items, improving performance by caching and only updating when needed.

3 UseNativeDriver

Optimize animations using the native driver for smoother transitions and reduced CPU overhead.

Pagination with FlatList

```
const [data, setData] = useState([]);
const [page, setPage] = useState(1);

const fetchData = async () => {
  const response = await fetch(`https://api.example.com/items?page=${page}`);
  const newData = await response.json();
  setData([...data, ...newData]); // Append new data
  setPage(page + 1);
};

return (
  <FlatList
    data={data}
    renderItem={({ item }) => <Text>{item.title}</Text>}
    onEndReached={fetchData} // Load more data when reaching the end
    onEndReachedThreshold={0.5} // Load new data when scrolled 50% near the bottom
  />
);
```

https://snack.expo.dev/@taitv/pagination-with-flatlist

React.memo with FlatList

```
const ListItem = React.memo(({ item }) => {
  return <Text>{item.title}</Text>;
});

return (
  <FlatList
    data={data}
    renderItem={({ item }) => <ListItem item={item} />}
    keyExtractor={(item) => item.id}
  />
);
```

https://snack.expo.dev/@taitv/reactmemo-with-flatlist?platform=android

Animated with useNativeDriver

```
const fadeAnim = useRef(new Animated.Value(0)).current;

useEffect(() => {
  Animated.timing(fadeAnim, {
    toValue: 1,
    duration: 500,
    useNativeDriver: true, // Kích hoạt native driver
  }).start();
}, []);

return <Animated.View style={{ opacity: fadeAnim }} />;
```

https://snack.expo.dev/@taitv/animated-with-usenativedriver?platform=android